PWA – Programmable Word Architecture

Andreas Skånning (s241123), Jonas Beck Jensen (s240324)
Mads Rudolph (s246132), Sigurd Hestbech Christiansen (s245534)

Marts 2026

Kursus 62711 – Design of Digital Systems

Andreas
s241123

Mads
s246132

Jonas
s240324

Sigurd
s245534

Gruppe 3

DTU Elektro
Diplomingeniør, Elektro

Contents

1 Introduktion **3**

2 Datapath Design **3**
2.1 Input- og outputsignaler . . . 3
2.2 Datapath Flow . . . 4

3 Register File **5**
3.1 Timing Diagram — Register Transfer . . . 5
3.2 D-Flip-Flop . . . 6
3.3 MUX2x1 . . . 6
3.4 8-bit Register . . . 7
3.5 RegisterR16 . . . 7
3.6 Destination Decoder . . . 8
3.7 MUX16x1x8 . . . 8
3.8 Register File Test . . . 9

4 Function Unit **9**
4.1 Eksempelberegninger . . . 10
4.2 ALU . . . 11
4.2.1 Full Adder . . . 12
4.3 Shifter . . . 14
4.4 Function Select . . . 15
4.5 MUX2x1x8 (MUXF) . . . 16
4.6 NegZero . . . 16

5 Datapath **17**
5.1 MUXB . . . 17
5.2 MUXD . . . 17
5.3 Datapath Test . . . 17

6 Konklusion **18**

A Simuleringslog **20**
A.1 Register File Timing Diagram – Simuleringslog . . . 20
A.2 Datapath Testbench – Simuleringslog . . . 20

1 Introduktion

I dette projekt har vi designet, simuleret og dokumenteret en komplet Programmable Word Architecture (PWA), som udgør grundstrukturen for en simpel datapath bestående af en Register File, en Function Unit (ALU + shifter) samt den nødvendige dekoder- og multiplexerlogik. Projektet er første del af en større modulopbygget udviklingsproces, hvor fokus ligger på at forstå, konstruere og verificere de centrale hardwarekomponenter i et digitalt system.
Formålet med arbejdet er dels at opnå en dybere forståelse af, hvordan en datapath fungerer på signalniveau, og dels at udvikle de enkelte moduler gennem strukturel VHDL og systematiske testbenches. Register File-modulet er ansvarligt for lagring og selektiv udlæsning af data til datapathen, mens Function Unit-enheden udfører de aritmetiske og logiske mikrooperationer, der styres via det 4-bit brede function select-signal. I projektet indgår også design af både timingdiagrammer, funktionsoversigter og verificering af signalflow mellem modulerne.
Hver komponent konstrueres, simuleres og dokumenteres, før den indgår i det samlede datapath-design. Resultatet er en fuldt fungerende PWA-datapath, der korrekt kan modtage, gemme, behandle og returnere data gennem de specificerede mikrooperationer.

2 Datapath Design

Datapath er den centrale enhed i vores PWA-arkitektur og forbinder alle de moduler, der er nødvendige for at lagre, transportere og behandle data. Det overordnede blokdiagram (se figur PWA-1 i opgavebeskrivelsen) viser, hvordan de tre hovedblokke — Register File, Function Unit og den tilhørende multiplexer-/dekoderlogik — er forbundet.

2.1 Input- og outputsignaler

Datapath-modulet har følgende eksterne signaler:

Signal	**Retning**	**Bredde**	**Beskrivelse**
CLK	in	1 bit	Clock-signal til synkron register-overførsel
RESET	in	1 bit	Asynkront reset af alle registre
RW	in	1 bit	Read/Write — aktiverer skrivning til Register File ved rising edge
DA(3:0)	in	4 bit	Destination Address — vælger hvilket register der skrives til
AA(3:0)	in	4 bit	A Address — vælger register til A_Data output
BA(3:0)	in	4 bit	B Address — vælger register til B_Data output
ConstantIn(7:0)	in	8 bit	Ekstern konstant til MUXB
MB	in	1 bit	Styrer MUXB: 0 = B_Data fra RF, 1 = ConstantIn
FS3–FS0	in	4 bit	Function Select — bestemmer mikrooperationen i Function Unit
Cin	in	1 bit	Carry-in til ALU'ens adder
DataIn(7:0)	in	8 bit	Ekstern data-input til MUXD
MD	in	1 bit	Styrer MUXD: 0 = F fra Function Unit, 1 = DataIn
Address_Out(7:0)	out	8 bit	A_Data fra Register File (bruges som adressebus)
Data_Out(7:0)	out	8 bit	B_Data direkte fra Register File
V	out	1 bit	Overflow-flag fra ALU
C	out	1 bit	Carry-flag fra ALU
N	out	1 bit	Negative-flag fra NegZero
Z	out	1 bit	Zero-flag fra NegZero

Table 1: Datapath input- og outputsignaler

2.2 Datapath Flow

Signalflowet gennem datapath kan beskrives i følgende trin:

1. **Register File → A_Data og B_Data (8 bit):** Register File modtager adresserne AA(3:0) og BA(3:0), som via to 16-til-1 multiplexere (MUX16x1x8) vælger de ønskede registerværdier. A_Data og B_Data er begge 8 bit brede og føres ud som operander.

2. **A_Data → Function Unit input A (8 bit):** A_Data føres direkte til Function Unit som operand A. Derudover føres A_Data ud som Address_Out.

3. **B_Data → MUXB → Function Unit input B (8 bit):** B_Data passerer gennem MUXB (MUX2x1x8), som styres af MB-signalet (1 bit). Hvis MB = 0, vælges B_Data; hvis MB = 1, vælges ConstantIn (8 bit). Outputtet Bus_B føres til Function Unit som operand B. Bemærk at Data_Out tages direkte fra B_Data (før MUXB).

4. **Function Unit — intern signalflow:**
   - FS(3:0) føres til Function Select (FSel), som genererer MF-signalet (1 bit) ved at AND'e FS3 og FS2.
   - JSel (4 bit) afledt af FS styrer ALU'ens operation. ALU modtager A og B (begge 8 bit) samt Cin (1 bit) og producerer J (8 bit) samt V og C (begge 1 bit).
   - HSel (2 bit) afledt af FS(1:0) styrer Shifterens operation. Shifteren modtager B (8 bit) og producerer H (8 bit).
   - MUXF (MUX2x1x8) styres af MF (1 bit) og vælger mellem J (ALU-output) og H (Shifter-output). Resultatet F (8 bit) sendes til NegZero og MUXD.
   - NegZero modtager F (8 bit) og genererer N- og Z-flagene (begge 1 bit).

5. **F → MUXD → D_Data (8 bit):** Resultatet F fra Function Unit passerer gennem MUXD (MUX2x1x8), som styres af MD-signalet (1 bit). Hvis MD = 0, vælges F; hvis MD = 1, vælges DataIn (8 bit). MUXD's output D_Data (8 bit) føres tilbage til Register File.

6. **D_Data → Register File (skrivning):** D_Data (8 bit) skrives til det register, som er valgt af DA(3:0) via Destination Decoder, men kun hvis RW = 1 og der er en opadgående clock-kant.

Tabel 2 opsummerer de interne signaler og deres bredder:

Signal	Bredde	Forbindelse
A_Data	8 bit	Register File → FU (A), Address_Out
B_Data	8 bit	Register File → MUXB
Bus_B	8 bit	MUXB → FU (B)
D_Data	8 bit	MUXD → Register File
J	8 bit	ALU → MUXF
H	8 bit	Shifter → MUXF
F	8 bit	MUXF → NegZero, MUXD
Load(15:0)	16 bit	Destination Decoder → RegisterR16
JSel	4 bit	FS(3:0) → ALU (styrer B-transformation og aritmetik/logik)
HSel	2 bit	FS(1:0) → Shifter
Cin	1 bit	Ekstern carry-in → ALU adder
MF	1 bit	FSel → MUXF

Table 2: Interne signaler i datapath med bredde og forbindelse

3 Register File

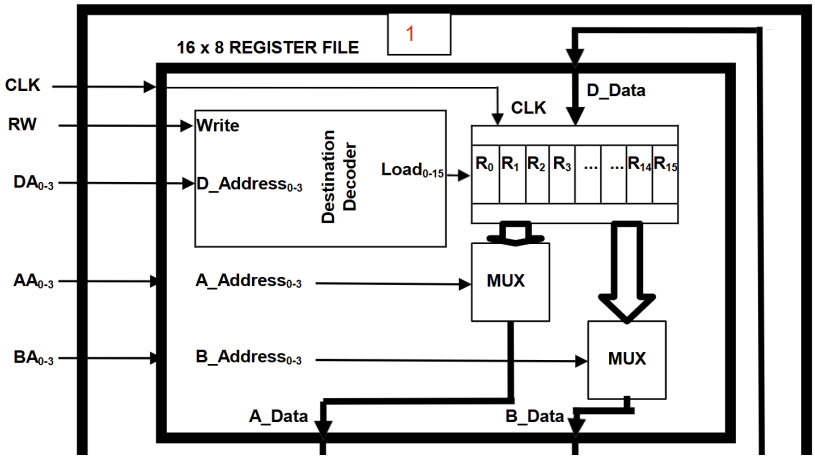

Figure 1: Register File top

Register File udgør den ene halvdel af Datapath, **Fig 1** viser Register File og de underliggende komponenter som er følgende:

- **Destination Decoder** - En 4x16 dekoder som modtager Write signal samt adressen på hvilket register som skal skrives til.
- **RegisterR16** - En register blok som indeholder 16 registre.
- **MUXA & MUXB** - To multiplexer som udvælger hvilke register værdier som skal skrives til output A_Data og B_Data.

Hovedopgaverne som Register File udfører er at lagre data fra Function Unit eller inputdata som kommer udefra datapath.

3.1 Timing Diagram — Register Transfer

For at illustrere, hvordan en skrivning til Register File foregår, beskrives her et eksempel med DA = 1010 (register R10).
Forløbet er som følger:

1. **Opsætningsfase:** Kontrolsignalerne sættes: RW = 1, DA = `"1010"`, og D_Data indeholder den ønskede værdi (fx `0x42`).
2. **Destination Decoder:** Da RW = 1, dekoder DD adressen DA = 1010 til Load(10) = 1, mens alle øvrige Load-signaler forbliver 0.
3. **Opadgående clock-kant:** Ved den opadgående flanke af CLK overføres D_Data til register R10, da Load(10) er aktiv. Alle andre registre beholder deres nuværende værdi, da deres respektive Load-signaler er 0.
4. **Efter clock-kanten:** R10 indeholder nu den nye værdi. Hvis AA eller BA peger på register 10, vil A_Data eller B_Data afspejle den opdaterede værdi.

Hvis RW = 0 i stedet, vil Destination Decoder producere Load = `0x0000` uanset DA-værdien, og ingen registre opdateres — selv ved en opadgående clock-kant.
Fig 2 viser simuleringen af dette forløb. Testbenchen anvender `assert`-statements til at verificere hvert trin automatisk. Den tilhørende testbench-kode og det fulde simuleringslog kan ses i **Appendix A.1**.

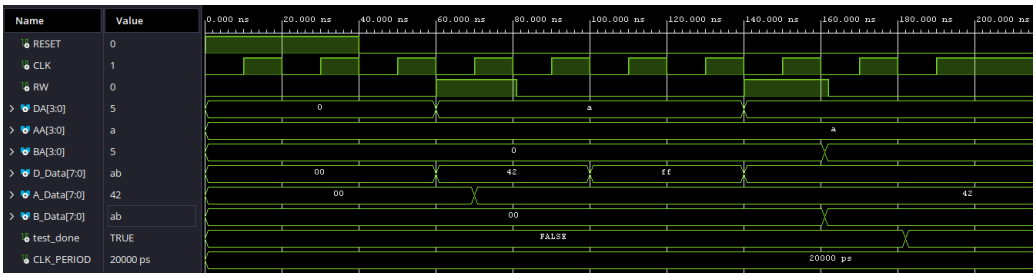

Figure 2: Timing diagram for skrivning til register R10 (DA = 1010)

3.2 D-Flip-Flop

Vi har programmeret en D-Flip-Flop, som vi senere anvendte til at konstruere registrene som RegisterR16 består af. D-Flip-Floppen er den eneste del af Datapath som er skrevet med behavioral VHDL-kode og skiller sig derfor lidt ud, men dette er nødvendigt, da en D-Flip-Flop skrevet via strukturel VHDL har problemer med at blive implementeret i hardware. **Fig 3** viser en testbench af flip-floppen som bekræfter flip-floppens funktionalitet, imens **tabel 3** forklarer hvad som simuleringen tester.

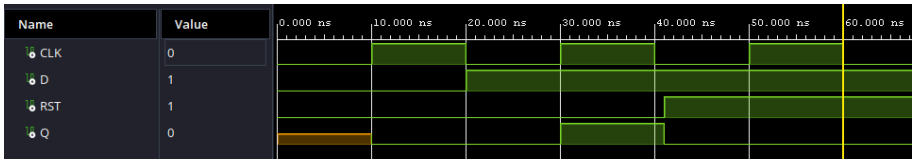

Figure 3: D-flip-flop simulering

Tidspunkt	**Test parametre**
10 ns	Input (`D`) overføres til output (`Q`) ved opadgående clock-kant
20 ns	Input (`D`) ændres før opadgående clock-kant for at teste at forrige input holdes helt indtil næste opadgående clock-kant
30 ns	Først nu opdateres output (`Q`) til det nye input (`D = 1`)
41 ns	Asynkront reset testes

Table 3: Gennemgang af flip-flop simuleringen

3.3 MUX2x1

Parallel-registrerene som RegisterR16 består af, skal have et enable-signal implementeret. Dette er nødvendigt, således at datapath selectivt kan vælge hvilket af de seksten registre i RegisterR16 som skal skrives til på et vilkårligt tidspunkt.

Dette enable-signal har vi implementeret med en 2 til 1 MUX, som sidder på indgangen af hver eneste D-flip-flop. MUX'erne, hvis output er forbundet til flip-floppernes input, vælger at sende enten det forrige output fra flip-floppen tilbage ind i dens input, eller en ny dataværdi som er modtaget fra `D_Data`-bussen.

Dette er en simpel måde at implementere et enable-signal på. Det tillader en D-flip-flop enten at gemme nye data, eller holde på den forrige data indtil der slukkes helt for systemet.

Fig 5 viser en diagramtegning af hvordan MUX'erne og D-flip-flopperne er forbundne. **Fig 4** viser en simulering af 2 til 1 MUX'en som bekræfter dens funktionalitet, imens **Tabel 4** forklarer hvad simuleringen tester.

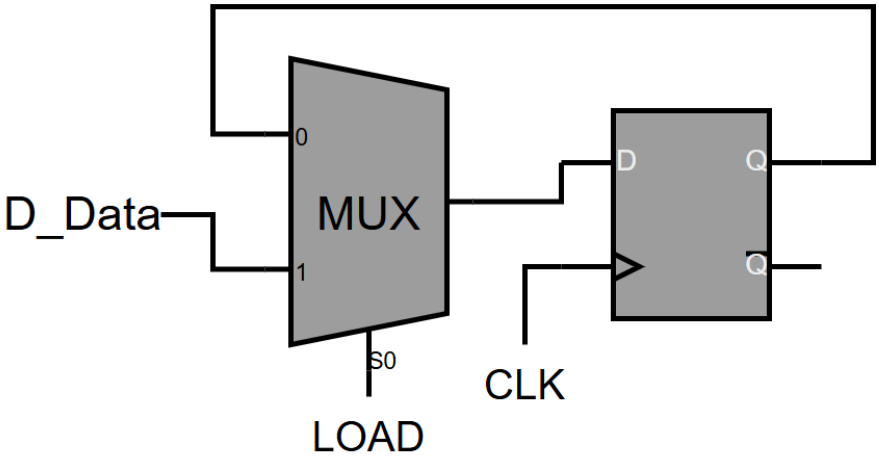

Figure 5: 1-bit register diagram

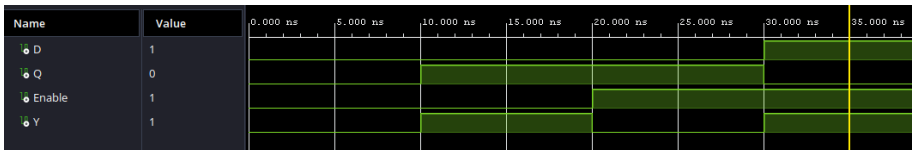

Figure 4: Simulering af 2 til 1 MUX

Tidspunkt	**Test parametre**
0 ns → 20 ns	Output (`Y`) følger input `Q`
20 ns → 35 ns	Output (`Y`) følger input `D`

Table 4: Gennemgang af MUX2x1 simuleringen

3.4 8-bit Register

8-bit parallel-registrerene som RegisterR16 indeholder, er sammensat af otte 1-bit registre som det på **Fig 5**. Hvert af de 1-bit registre modtager et enkelt bit fra D_Data-bussen, som varierer fra indeks 0 til 7.

3.5 RegisterR16

RegisterR16 er som beskrevet i de forrige afsnit en blok af seksten parallel-registre. Hvert register er opbygget af otte 1-bit registre, som alle benytter den samme MUX-baserede enable-funktionalitet fra **Fig 5**.

Fig 6 viser en simulering af RegisterR16, som bekræfter dets funktionalitet. **Tabel 5** forklarer, hvad simuleringen tester.

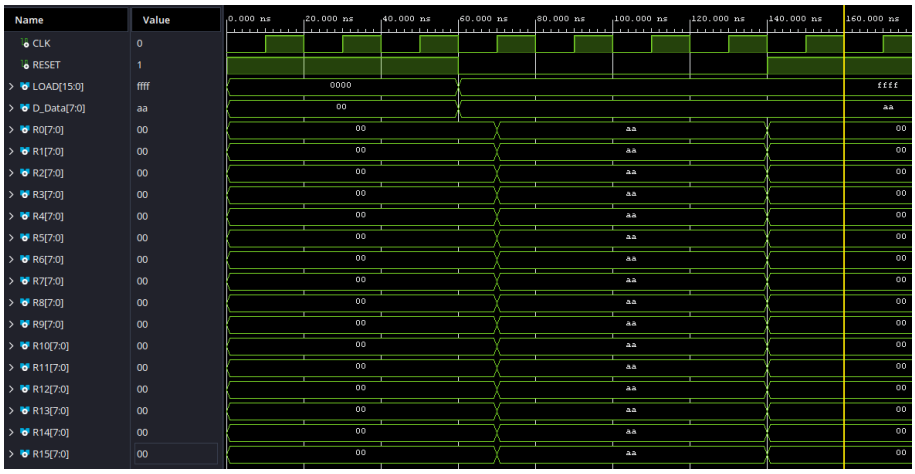

Figure 6: Simulering af RegisterR16

Tidspunkt	Test parametre
0 ns → 65 ns	Reset har været høj så alle registre er nul inden opadgående clock-kant
65 ns → 140 ns	LOAD sættes højt for alle registre og D_Data gemmes i alle registre
140 ns	alle registre resetes igen

Table 5: Gennemgang af RegisterR16 simuleringen

3.6 Destination Decoder

Destination Decoder er en 4 til 16 decoder, som udvælger hvilket register der skal loades data ind i, i register-blokken RegisterR16.

Ydermere har Destination Decoder et enable-signal kaldet RW (Read Write). Hvis ikke dette signal er højt, skrives der ikke til nogen registre.

Fig 7 viser en simulering af Destination Decoder, som bekræfter dets funktionalitet. **Tabel 6** forklarer, hvad simuleringen tester.

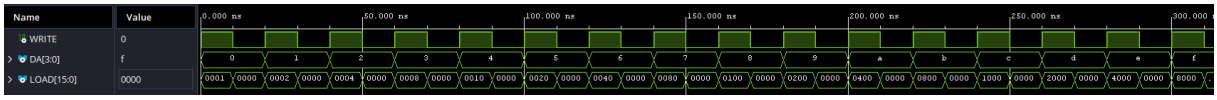

Figure 7: Simulering af DestinationDecoder

Tidspunkt	Test parametre
0 ns → 310 ns	Alle udgange fra decoderen testes for både korrekt aflæsning af data-adressen og for at bekræfte, at `RW` aktiverer de relevante udgange.

Table 6: Gennemgang af DestinationDecoder-simuleringen

3.7 MUX16x1x8

Både MUXA og MUXB består af en 16 til 1-byte MUX. Disse multiplexere er hver konstrueret af otte 16 til 1-bit MUX'er.

Fig 8 viser en simulering af MUX16x1x8, som bekræfter dets funktionalitet. **Tabel 7** forklarer, hvad simuleringen tester.

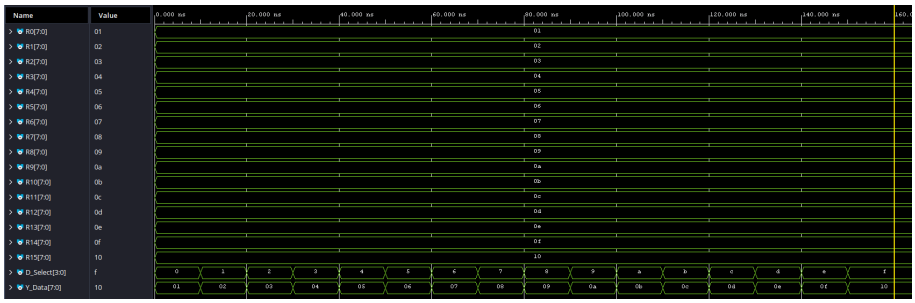

Figure 8: Simulering af MUX16x1x8

Tidspunkt	Test parametre
0 ns → 160 ns	MUX16x1x8 skifter mellem registerværdierne R0–R16 baseret på `D_Select`, med et skifte hver 10 ns

Table 7: Gennemgang af MUX16x1x8 simuleringen

3.8 Register File Test

Vi har testet hele register-file i en samlet simulering, som kan ses i **Fig 9**. Dette gøres for at bekræfte, at alle underdelene fungerer koblet sammen.
Tabel 8 forklarer, hvad simuleringen tester.

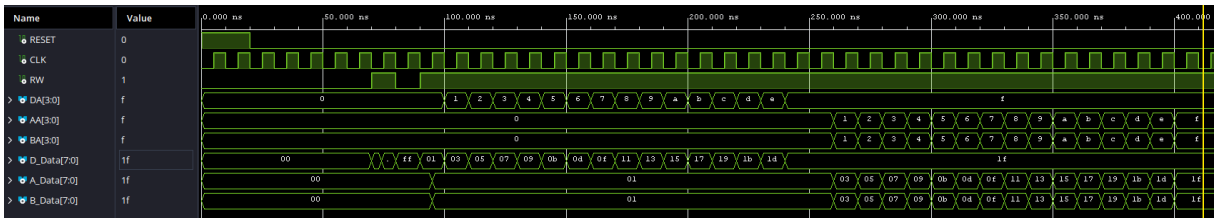

Figure 9: Simulering af hele RegisterFile

Tidspunkt	Test parametre
0 ns → 70 ns	Reset aktiveres for at sikre et klart udgangspunkt.
70 ns → 90 ns	Test af at indlæsning kun sker ved den opadgående clock-kant, samt at `Write`-signalet fungerer som forventet.
90 ns → 250 ns	Indlæsning af en unik værdi i hvert register.
250 ns → 410 ns	Skiftevis udskrift af alle registerværdier på `A_Data` og `B_Data`.

Table 8: Gennemgang af RegisterFile-simuleringen

4 Function Unit

Function Unit udfører mikro-operationer på operanderne A og B. Enheden kan udføre både logiske og aritmiske operationer samt right shift og left shift på B operanden.
Function Unit er styret af signalet FS(3:0) (function select), som bestemmer hvilken mikro-operation der udføres. Resultatet af den valgte mikro-operation F(7:0) føres herefter ind i MUXD.
Signalerne V, C, Z og N er flag der bliver sat ved forskellige forhold under operationen:

- V flaget kommer fra ALU'en og går højt hvis vi har aritmisk overflow.
- C flaget kommer fra ALU'en og går højt hvis vi har en **carry out** på **C(8)**.

- Z flaget går højt hvis resultatet på F(7:0) er lig 0.
- N flaget går højt hvis F(7:0) er et negativt resultat dvs. F(7) = N.

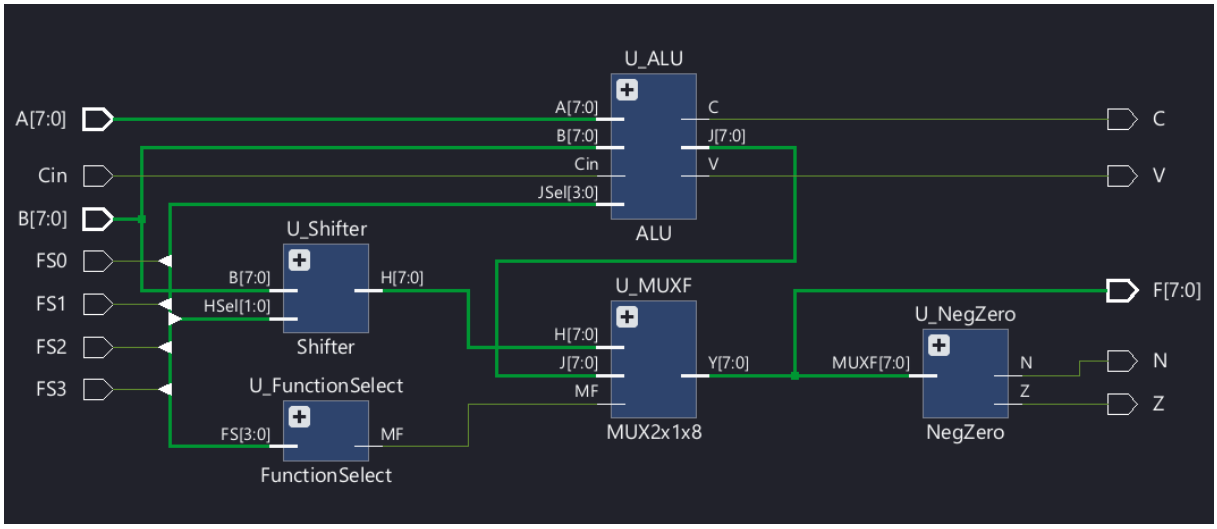

Figure 10: Function Unit blok diagram

4.1 Eksempelberegninger

For at demonstrere Function Unit's funktionalitet er der i **Tabel 9** vist konkrete eksempler på addition, subtraktion (2's komplement), XOR og shift left med de tilhørende styresignaler og flag-værdier.

#	FS	J_Sel	H_Sel	MF	A	B	F	V	C	N	Z
Addition (A + B), FS = 0010											
1	0010	0010	XX	0	03h	02h	05h	0	0	0	0
2	0010	0010	XX	0	7Fh	01h	80h	1	0	1	0
3	0010	0010	XX	0	FFh	01h	00h	0	1	0	1
Subtraktion ($A + \overline{B} + 1$), FS = 0101											
4	0101	0101	XX	0	08h	03h	05h	0	1	0	0
5	0101	0101	XX	0	03h	08h	FBh	0	0	1	0
6	0101	0101	XX	0	80h	01h	7Fh	1	1	0	0
XOR ($A \oplus B$), FS = 1010											
7	1010	1X10	XX	0	AAh	55h	FFh	0	0	1	0
Shift Left (sl B), FS = 1110											
8	1110	XXXX	10	1	–	55h	AAh	–	–	1	0

Table 9: Eksempelberegninger for Function Unit med flag-værdier. V og C er kun meningsfulde når MF = 0 (ALU-output valgt).

Bemærkninger til tabellen:

- **Eksempel 2:** $127 + 1 = 128$, men i signed 8-bit er $128 = -128$ (overflow). V = 1 da carry ind i MSB $\neq$ carry ud af MSB.
- **Eksempel 3:** $-1 + 1 = 0$. Carry out C = 1, men intet overflow (V = 0). Z = 1 da resultatet er nul.
- **Eksempel 5:** $3 - 8 = -5$ (FBh i 2's komplement). C = 0 indikerer borrow.
- **Eksempel 6:** $-128 - 1 = -129$, som ikke kan repræsenteres i signed 8-bit. V = 1 (overflow).
- **Eksempel 8:** Ved shift left er V og C fra ALU'en ikke relevante, da MF = 1 vælger shifterens output.

4.2 ALU

ALU modulet står for at udføre både aritmiske og logiske operationer. De aritmiske operationer er udført af en 8x1-bit full adder som er beskrevet i næste afsnit. De logiske operationer følger den sandhedstabel der er opgivet i opgavebeskrivelsen. Vi kan med JSel(3 downto 0) som styresignaler styre hvilken af de 4 logiske operationer ALU'en udfører.
Hver af de fire logiske funktioner (A OR B, A AND B, A XOR B og NOT A) genereres parallelt. JSel-signalet fungerer herefter som et selektionssignal, der bestemmer, hvilken af de logiske outputs der skal sendes videre til J-bussen. I praksis sker dette ved, at JSel-bittene aktiverer én af de fire logiske veje gennem AND-gates, hvorefter alle disse samles i en OR-gate.
Det betyder, at kun den funktion, som matcher mønsteret på JSel, bliver ført ud.
Figuren nedenfor viser strukturen: de fire logiske operationer beregnes først, og JSel-bittene bruges derefter til at vælge det aktive resultat.

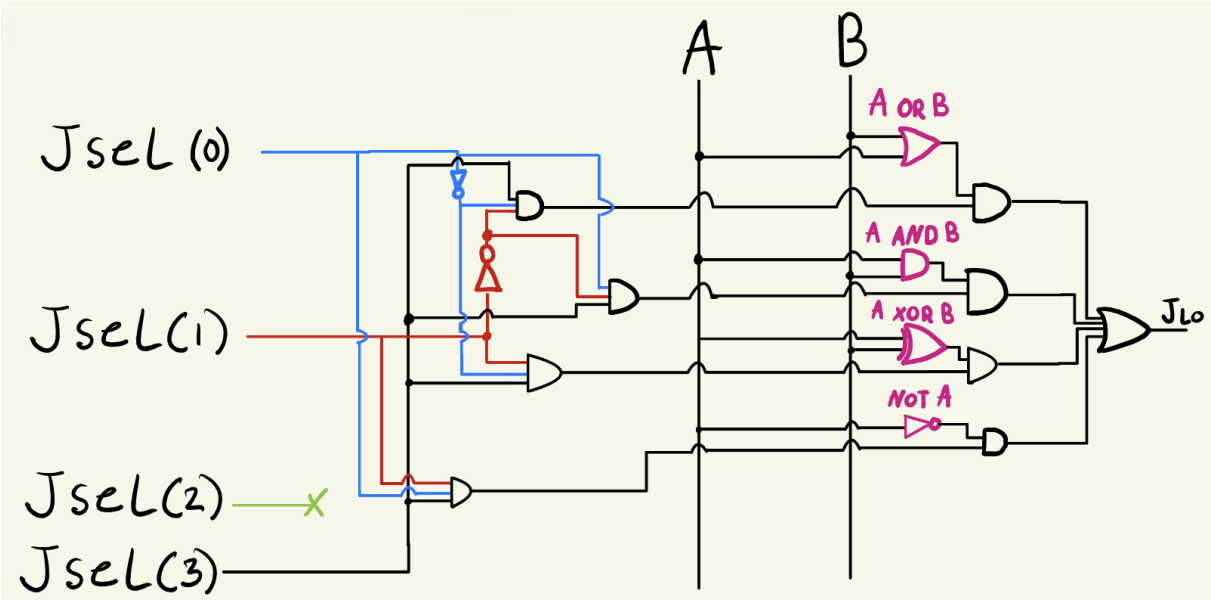

Figure 11: ALU Logic operation

De aritmetiske operationer i ALU'en er implementeret direkte ud fra funktionsdiagrammet, som blev gennemgået i Lesson 3. Diagrammet angiver, hvordan hver 1-bit full-adder skal forsynes med de korrekte signaler for at kunne udføre addition og subtraction i 2's komplement. Vi har taget strukturen fra diagrammet og omsat den én-til-én til VHDL i Vivado.

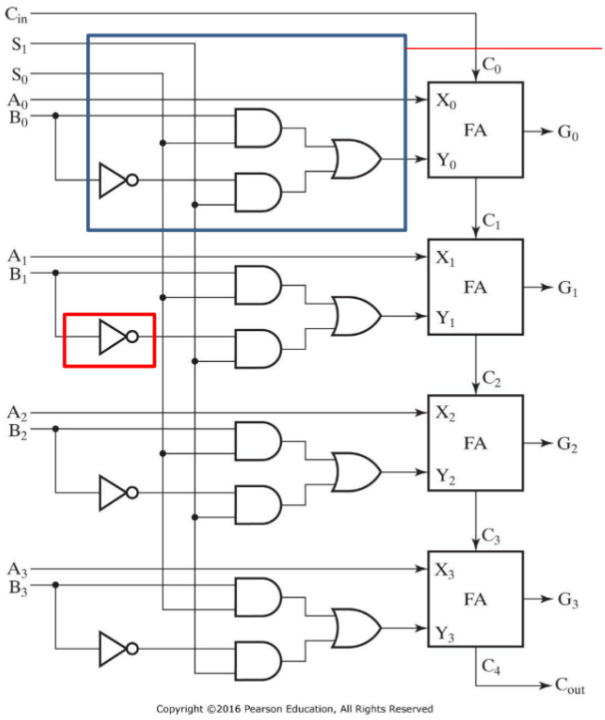

Figure 12: Funktionsdiagram for aritmetiske funktioner for ALU

Til sidst har vi samlet de aritmetiske og logiske operationer, som, de bliver valgt JSel(3), som styre hvilken operation bliver lukket igennem.
Nedenfor ses testbench af ALU'en og det er tydeligt at se at den opfører sig som forventet.

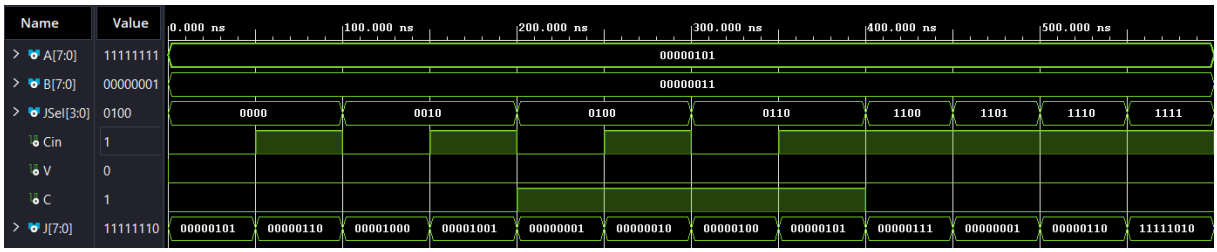

Figure 13: ALU TB

4.2.1 Full Adder

8-bit full adderen er sat sammen af 8x1-bit full adders der hver består af to half adders. måden den fungerer på er at ved det pågældende bit nummer (i), tages den binære værdi af operanderne A og B og føres ind på indgangene i den første half adder. For at vi i resultatet på samme bitnummer S(i) får et logisk 1 kræver det at enten kun en af operanderne er logisk 1 eller at begge operander samt carry in (Ci) er logisk 1. og for at vores carry out (Ci+1) bliver logisk 1 skal 2 eller 3 af A, B og Ci være logisk 1.

Nedenfor ses det logiske design af 1-bit full-adderen. Designet er inspireret af full adder designet fra slideshow 62711_lesson2_f2026 slide 5.

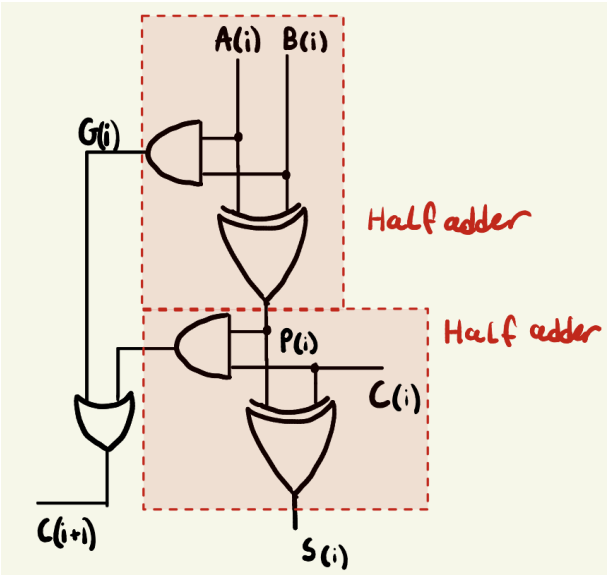

Figure 14: 1bit full adder

Herunder ses implementeringen af 8x1-bit full-adders med en enkelt 1-bit full adder expanderet så man kan se det logiske indhold.

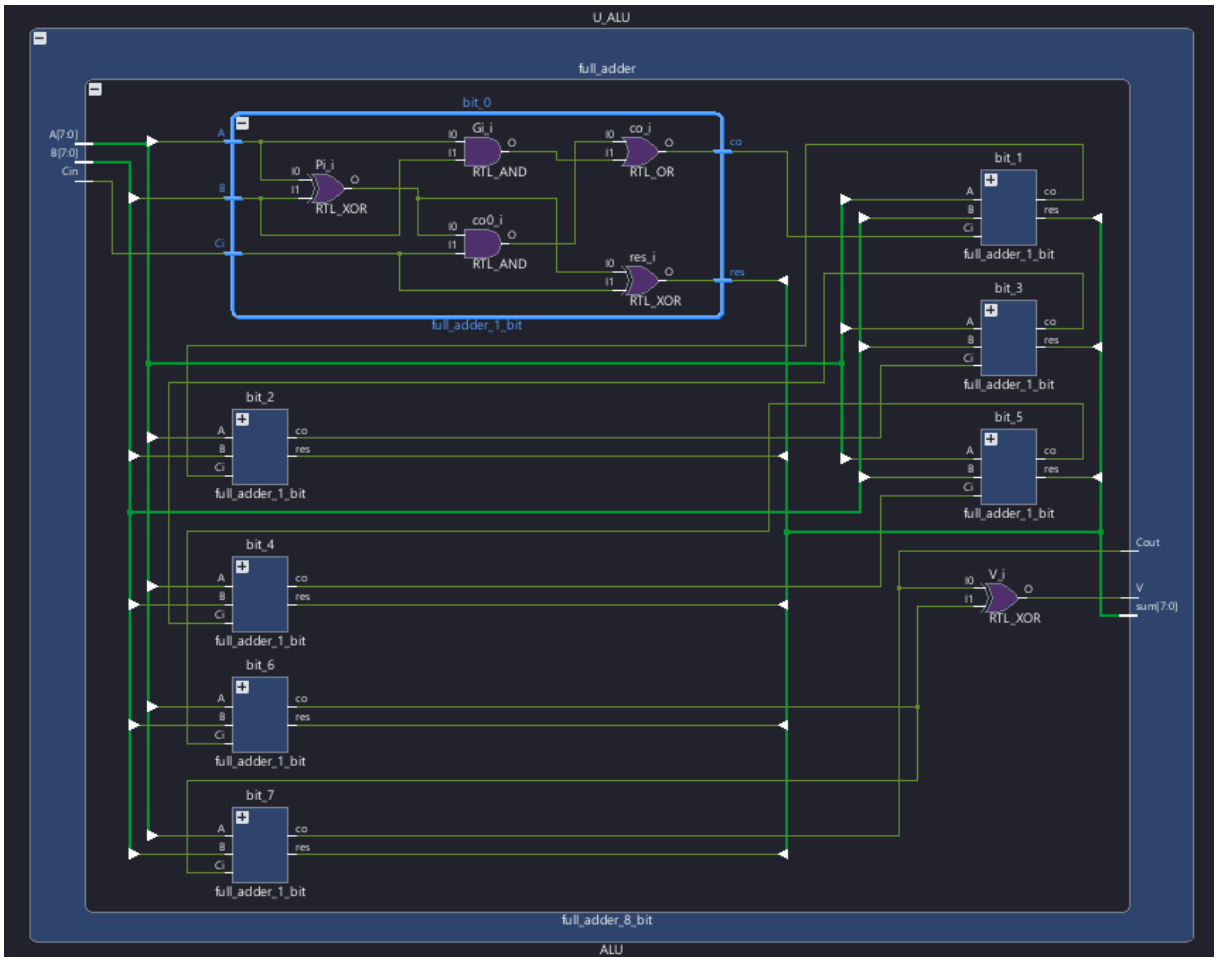

Figure 15: 8 bit full adder funktions diagram

Efter at have implementeret 8x1-bit full-adderen har vi lavet en test bench og vi kan konkludere at den virker efter hensigten.

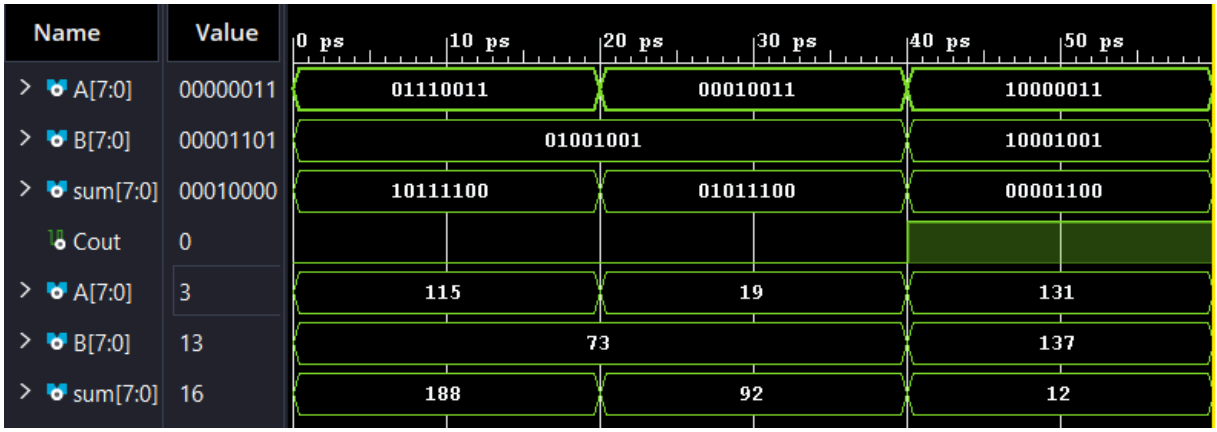

Figure 16: Full-adder TB

4.3 Shifter

Shifteren er designet ud fra vores eget funktionsdiagram og opfylder kravene i opgavebeskrivelsen. HSel-signalernes to bit føres ind i en XOR-gate, som sikrer, at der kun sker et shift, når bittene er forskellige. Når de er

ens, forbliver Htemp = 0, og der udføres derfor intet shift.
For at opnå dette inverteres HTemp og AND'es med den originale B-værdi. Når HSel-bittene er forskellige, bliver Htemp = 1, og dette signal AND'es med både HSel(0) og HSel(1). På den måde dannes to enable-signaler, sr og sl, som hver især aktiverer henholdsvis et højre- eller venstreshift. Disse enable-signaler trækkes ud til 8 individuelle porte, så hele 8-bit-bussen behandles korrekt og alle bit bliver shiftet ens. Selve shiftet sker ved at "klistre" et bit ind på enten MSB eller LSB, mens resten af bittene flyttes én position i den ønskede retning. Til sidst samles både det shiftede og det ikke-shiftede signal i en OR-gate. Da kun én betingelse kan være opfyldt ad gangen, sikrer OR-gaten, at det korrekte resultat altid fremkommer på output H, samtidig med at alle funktioner kan deles om samme udgang.

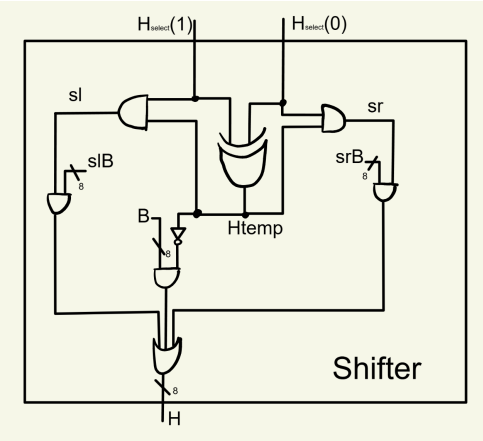

Figure 17: Shifter funktionsdiagram

Efter at have implementeret vores shifter-design i Vivado, har vi udarbejdet en testbench for at verificere, at modulet fungerer som forventet.

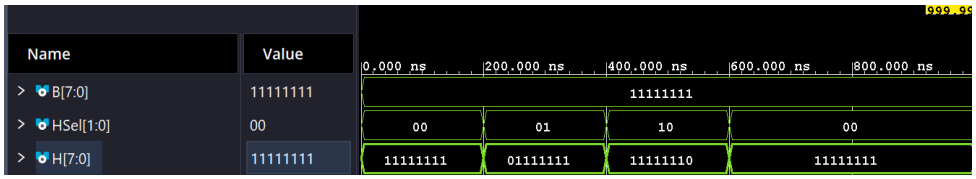

Figure 18: Test bench for shifter

Resultaterne fra simuleringen bekræfter, at designet opfører sig korrekt under alle testede betingelser.

4.4 Function Select

Function-select-signalet bruges til at generere MF-signalet. MF dannes ved at AND'e FS(3) og FS(2), hvilket sikrer, at MF kun bliver aktivt for de funktioner, hvor begge bit er sat. Dette signal anvendes efterfølgende til at vælge mellem ALU'ens og shifterens output.

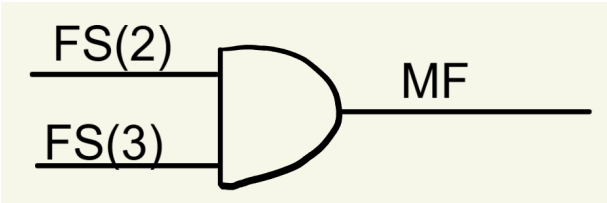

Figure 19: Funktionsdiagram for Function Select

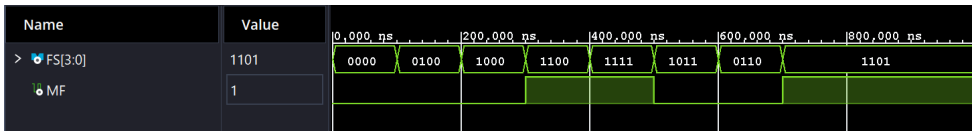

Figure 20: Testbench for function select

Testbenchen for function select viser at fungere som forventet.

4.5 MUX2x1x8 (MUXF)

MUXF bruges til at vælge, hvilket resultat der skal sendes videre til MUXD. Her kan resultatet komme enten fra ALU'en eller shifteren. Det videresendte resultat bestemmes af MF-signalet: Når MF = 1, vælges shifterens output, og når MF = 0, vælges ALU'ens output. Det valgte resultat sendes desuden videre til NegZero-modulet.

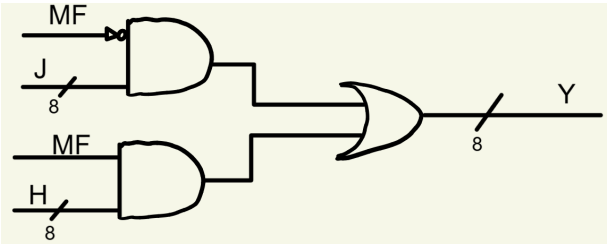

Figure 21: Funktionsdiagram for MUXF

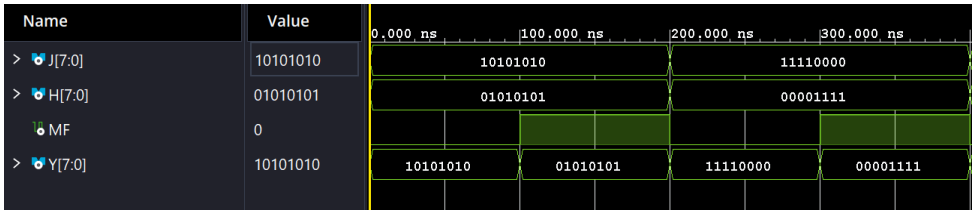

Figure 22: Test bench for MUXF

Test bench for MUXF viser, at den fungere som forventet.

4.6 NegZero

NegZero-modulet modtager resultatet fra MUXF og sætter N- og Z-flagene for at indikere, om resultatet er negativt eller lig med 0. Det negative flag bestemmes ved at undersøge MSB'en af F: Hvis dette bit er 1, tolkes resultatet som et signed negativt tal, og N-flaget sættes. Zero-detektionen sker ved at AND'e alle bit i resultatet. Det samlede signal inverteres derefter, hvilket giver en logisk 1 på Z-flaget, når hele resultatet er 0.

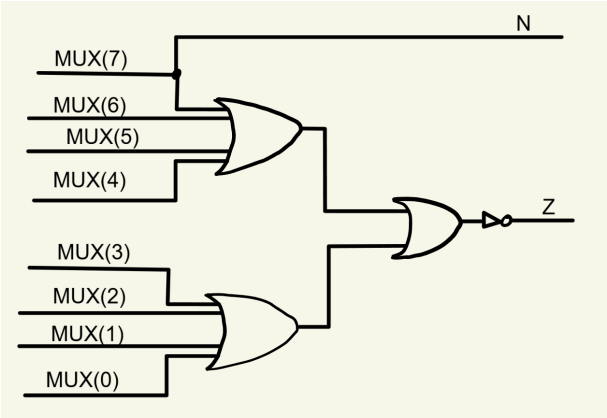

Figure 23: Funktionsdiagram for NegZero

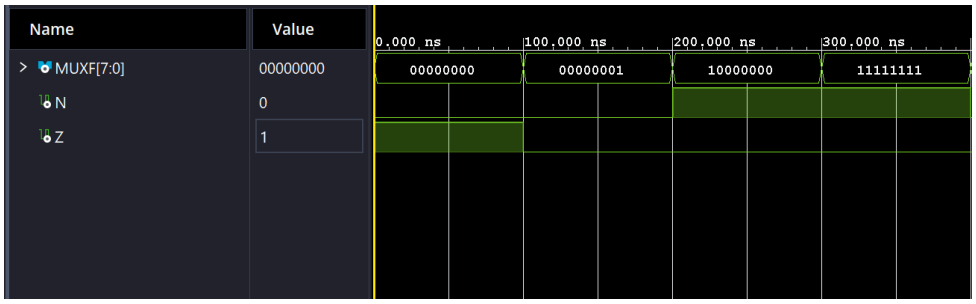

Figure 24: Test bench for NegZero

Test bench for NegZero viser at den fungere som forventet.

5 Datapath

5.1 MUXB

MUXB er en 8-bit 2-to-1 multiplexer, der vælger hvilken operand der føres ind som B-input til Function Unit. Multiplexeren styres af kontrolsignalet MB, som bestemmer om data fra Register File eller en ekstern konstant bruges som operand. Når MB er 0, vælges B_Data fra Register File, som er den værdi der læses fra det register som er valgt af B-adressen. Når MB er 1, vælges ConstantIn, hvilket gør det muligt at bruge en immediate konstant direkte i en mikrooperation. MUXB er implementeret som en instans af MUX2x1x8 komponenten, hvor J er forbundet til B_Data, H til ConstantIn, og MF til MB signalet. Outputtet Bus_B føres videre som B-operand til Function Unit.

5.2 MUXD

MUXD er en 8-bit 2-to-1 multiplexer, der vælger hvilken data der skrives tilbage til Register File som D_Data. Multiplexeren styres af kontrolsignalet MD, som bestemmer om resultatet fra Function Unit (F) eller ekstern data (DataIn) føres videre. Når MD er 0, vælges resultatet F fra Function Unit, hvilket bruges ved aritmetiske og logiske operationer. Når MD er 1, vælges DataIn, som gør det muligt at loade ekstern data direkte ind i registrene. MUXD er implementeret som en instans af MUX2x1x8 komponenten, hvor J er forbundet til F_Out, H til DataIn, og MF til MD signalet.

5.3 Datapath Test

Vi har lavet en samlet testbench med 19 automatiserede tests, som verificerer hele Datapath mod funktionstabellen i opgavebeskrivelsen. Testbenchen anvender `assert`-statements, så hvert trin enten bestås eller fejler med en tydelig besked i konsollen. **Fig 25** viser simuleringen, og det fulde konsol-output kan ses i **Appendix A.2**.

De 19 tests dækker følgende:

Test	Beskrivelse	FS / Funktion
1	Reset nulstiller alle registre	–
2	Indlæsning via DataIn (MD = 1)	–
3	F = A (transfer)	0000
4	F = A + 1	0001
5	F = A + B	0010
6	F = A + B + 1	0011
7	F = A + $\overline{B}$	0100
8	F = A − B (A + $\overline{B}$ + 1)	0101
9	F = A − 1	0110
10	F = A OR B	1000
11	F = A AND B	1001
12	F = A XOR B	1010
13	F = NOT A	1011
14	F = B (shifter pass-through)	1100
15	F = sr B (shift right)	1101
16	F = sl B (shift left)	1110
17	MUXB vælger ConstantIn (MB = 1)	0010
18	Flag-check (V, C, N, Z)	0010
19	Write-back loop: RF → FU → RF	0010

Table 10: Oversigt over de 19 Datapath-tests

Alle 19 tests bestod uden fejl, hvilket bekræfter at den samlede Datapath korrekt udfører samtlige mikro-operationer fra funktionstabellen, at MUXB og MUXD vælger de rigtige datakilder, at flag-signalerne (V, C, N, Z) genereres korrekt, samt at write-back til Register File fungerer som forventet.

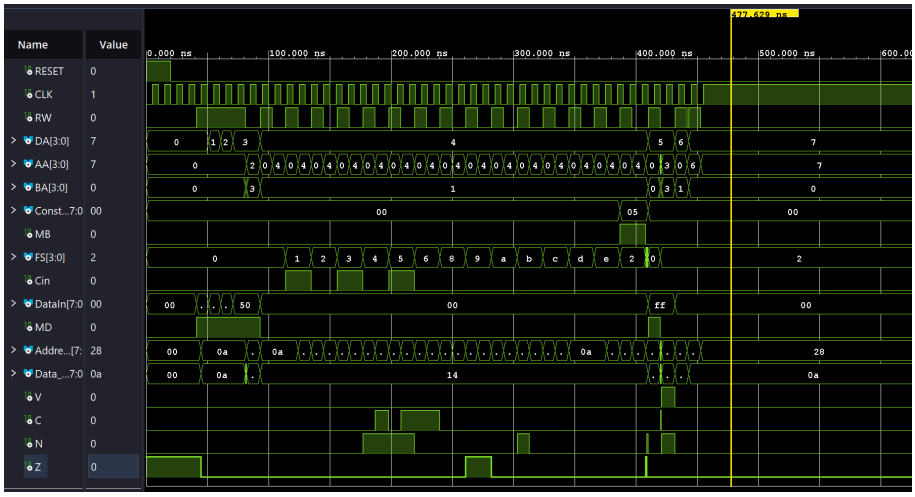

Figure 25: Simulering af Datapath

6 Konklusion

I dette projekt har vi udviklet og testet en fuldt fungerende Programmable Word Architecture (PWA), som består af en Register File, en Function Unit samt de nødvendige multiplexere og dekodere. Undervejs har vi designet hver komponent ud fra de krav, der er angivet i opgavebeskrivelsen, og verificeret deres funktion gennem målrettede testbenches.

Resultaterne viser, at Register File'en korrekt kan lagre, opdatere og udlæse data baseret på de styrende signaler. Function Unit'en udfører alle specificerede mikrooperationer — herunder addition, subtraction, logiske funktioner og shift-operationer — og genererer de korrekte flag for overflow, carry, zero og negative resultater. Samspillet mellem modulerne fungerer som forventet, og den samlede datapath lever op til kravene i projektets specifikation.

Projektet har givet os en solid forståelse af, hvordan en datapath er opbygget, hvordan signalerne flyder mellem modulerne, og hvordan digital hardware kan struktureres, testes og dokumenteres systematisk. Det færdige design opfylder alle opstillede funktionelle krav og danner et stabilt grundlag for det videre arbejde i de kommende projektfaser.

Modulansvar

Modul (VHDL + TB)	Ansvarlig	Subgruppe
D-Flip-Flop, MUX2x1	Sigurd	1
Register8bit, RegisterR16	Sigurd	1
Destination Decoder	Sigurd	1
MUX16x1x8, RegisterFile	Sigurd	1
RegisterFile TB	Mads	1
ALU (logik + aritmetik)	Andreas	2
Full Adder (1-bit + 8-bit)	Andreas	2
Shifter	Jonas	2
Function Select, NegZero	Jonas	2
MUX2x1x8 (MUXF)	Jonas	2
FunctionUnit	Andreas & Jonas	2
Datapath (top-level wiring)	Mads	Fælles
Datapath TB	Mads	Fælles
Rapport	Alle	Fælles

Table 11: Fordeling af modulansvar

Tidsforbrug

Navn	Studienr.	Timer
Andreas Skånning	s241123	8
Jonas Beck Jensen	s240324	8
Mads Rudolph	s246132	7
Sigurd Hestbech Christiansen	s245534	8
Total		**35**

Table 12: Tidsforbrug per gruppemedlem

A Simuleringslog

A.1 Register File Timing Diagram – Simuleringslog

Herunder ses konsol-outputtet fra Vivado xsim for `RegisterFile_timing_tb`. Testbenchen verificerer skrivning til register R10 (DA = 1010) som beskrevet i **Afsnit 3.1**.
Bemærk: Denne testbench er genereret ved hjælp af AI (Claude).

```
Time resolution is 1 ps

Note: PHASE 1: Reset complete, R10 = 0x00
Time: 60 ns  Iteration: 0
Process: /RegisterFile_timing_tb/STIM

Note: PHASE 2: Write successful, R10 = 0x42
Time: 81 ns  Iteration: 0
Process: /RegisterFile_timing_tb/STIM

Note: PHASE 3: Write blocked (RW=0), R10 still = 0x42
Time: 121 ns  Iteration: 0
Process: /RegisterFile_timing_tb/STIM

Note: PHASE 4: R5 = 0xAB, R10 still = 0x42
Time: 162 ns  Iteration: 0
Process: /RegisterFile_timing_tb/STIM

Note: TIMING DIAGRAM TB: ALL PHASES PASSED
Time: 182 ns  Iteration: 0
Process: /RegisterFile_timing_tb/STIM
```

Vivado xsim konsol-output

Alle fire faser bestod uden fejl, hvilket bekræfter at Register File'en korrekt håndterer skrivning, blokering ved RW = 0, og isolation mellem registre.

A.2 Datapath Testbench – Simuleringslog

Herunder ses konsol-outputtet fra Vivado xsim for `Datapath_tb`. Testbenchen verificerer alle 16 mikro-operationer fra funktionstabellen samt MUXB, flag-signaler og write-back som beskrevet i **Afsnit 5.3**.

```
Time resolution is 1 ps

Note: TEST 1: Reset clears all registers
Note: TEST 1 PASSED                          Time: 31 ns

Note: TEST 2: Load registers via DataIn
Note: TEST 2 PASSED                          Time: 83 ns

Note: TEST 3: F = A (transfer)
Note: TEST 3 PASSED                          Time: 104 ns

Note: TEST 4: F = A + 1
Note: TEST 4 PASSED                          Time: 125 ns

Note: TEST 5: F = A + B
Note: TEST 5 PASSED                          Time: 146 ns

Note: TEST 6: F = A + B + 1
Note: TEST 6 PASSED                          Time: 167 ns

Note: TEST 7: F = A + not B
Note: TEST 7 PASSED                          Time: 188 ns

Note: TEST 8: F = A - B
Note: TEST 8 PASSED                          Time: 209 ns

Note: TEST 9: F = A - 1
Note: TEST 9 PASSED                          Time: 230 ns

Note: TEST 10: F = A OR B
Note: TEST 10 PASSED                         Time: 251 ns

```

```
Note: TEST 11: F = A AND B
Note: TEST 11 PASSED                          Time: 272 ns

Note: TEST 12: F = A XOR B
Note: TEST 12 PASSED                          Time: 293 ns

Note: TEST 13: F = NOT A
Note: TEST 13 PASSED                          Time: 314 ns

Note: TEST 14: F = B (shift pass-through)
Note: TEST 14 PASSED                          Time: 335 ns

Note: TEST 15: F = sr B (shift right)
Note: TEST 15 PASSED                          Time: 356 ns

Note: TEST 16: F = sl B (shift left)
Note: TEST 16 PASSED                          Time: 377 ns

Note: TEST 17: MUXB selects ConstantIn
Note: TEST 17 PASSED                          Time: 398 ns

Note: TEST 18: Flag checks
Note: TEST 18 PASSED                          Time: 422 ns

Note: TEST 19: Write-back loop RF -> FU -> RF
Note: TEST 19 PASSED                          Time: 454 ns

Note: ALL TESTS PASSED                        Time: 454 ns
```

Vivado xsim konsol-output – Datapath_tb

Alle 19 tests bestod uden fejl, hvilket bekræfter at den samlede Datapath fungerer korrekt i henhold til specifikationen.